

[50] Exact Cardinality Query Optimization with Bounded Execution Cost

요약

본 논문은 Immanuel Trummer가 2019년 SIGMOD에 발표한 논문으로, ECQO (Exact Cardinality Query Optimization)의 개념을 처음 제시한 기념비적 작업이다. 이 논문의 핵심은 쿼리 최적화 과정에서 정확한 카디널리티(결과 개수)를 알 수 있다면, 더욱 효율적인 실행 계획을 선택할 수 있다는 직관에서 출발한다.

전통적인 쿼리 최적화는 주로 통계 정보를 기반으로 카디널리티를 추정한다. 하지만 이러한 추정은 종종 부정확하며, 특히 유사성 검색과 같은 복잡한 쿼리에서는 추정 오류가 심각할 수 있다. Trummer는 이 문제를 해결하기 위해, 제한된 실행 비용(bounded execution cost) 내에서 정확한 카디널리티를 결정할 수 있는 방법을 제시했다.

ECQO의 핵심 아이디어는 다음과 같다. 완전한 쿼리 실행 대신, 부분적 실행을 통해 결과의 크기를 정확히 파악할 수 있다면, 이를 기반으로 최적의 실행 계획을 선택할 수 있다는 것이다. 예를 들어, 조인(join) 쿼리에서 각 조인 단계 후의 정확한 카디널리티를 알면, 다음 조인의 순서를 동적으로 최적화할 수 있다.

이 접근법은 특히 유사성 검색과 같은 새로운 쿼리 유형에 매우 적합하다. 유사성 쿼리의 결과 크기는 거리 임계값에 매우 민감하며, 이를 정확하게 예측하는 것은 매우 어렵다. 그러나 실제 데이터에 대해 부분적으로 쿼리를 실행해보면, 정확한 결과 크기를 빠르게 파악할 수 있다.

본 논문은 Exqutor의 직접적인 영감이 되었으며, Exqutor는 이 개념을 한 단계 더 발전시켜 머신러닝을 사용한 카디널리티 추정 방법을 제시한다. ECQO는 원리적 접근(principled approach)을 제시했고, Exqutor는 실무적 구현을 제공한 것이다.

상세분석

50.1 카디널리티 추정의 전통적 문제점

전통적인 쿼리 최적화에서 카디널리티 추정은 매우 중요한 역할을 한다. 쿼리 옵티마이저는 추정된 카디널리티를 기반으로 실행 계획을 선택하기 때문이다. 예를 들어, 조인 쿼리에서 A 테이블과 B 테이블을 조인할 때, A를 먼저 필터링한 후 B와 조인하는 것이 좋은지, 아니면 B를 먼저 필터링하는 것이 좋은지는 각 단계 후의 카디널리티에 따라 결정된다.

전통적인 카디널리티 추정은 다음과 같은 문제점을 가진다. 첫째, 통계 정보에 기반하고 있다. 이는 실제 데이터 분포와 다를 수 있다. 예를 들어, 테이블의 히스토그램(histogram)이 실제 값의 분포를 완벽하게 반영하지 못할 수 있다.

둘째, 독립성 가정(independence assumption)을 기반으로 한다. 여러 조건이 결합될 때, 일반적으로 각 조건이 독립적이라고 가정하고 확률을 곱한다. 하지만 현실에서 여러 조건은 종종 상관관계가 있다. 예를 들어, 도시(city) 필터와 우편번호(zip code) 필터가 결합되었을 때, 이 두 조건은 독립적이지 않으므로 확률 곱셈은 부정확하다.

셋째, 복잡한 쿼리에서는 추정 오류가 누적된다. 여러 조인 단계를 거치면서, 각 단계의 추정 오류가 누적되어 최종 추정값의 정확도가 심각하게 악화될 수 있다.

넷째, 새로운 쿼리 유형(예: 유사성 검색, 그래프 쿼리)에 대해서는 통계 기반의 추정 방법이 적용되기 어렵다. 벡터 거리의 분포나 유사성 쿼리의 결과 크기를 전통적인 통계 방법으로 추정하는 것은 매우 어렵다.

50.2 ECQO의 핵심 개념: 부분 실행(Partial Execution)

ECQO의 혁신적인 아이디어는 완전한 쿼리 실행 없이도 정확한 카디널리티를 파악할 수 있다는 것이다. 이는 부분 실행(partial execution)을 통해 가능하다.

부분 실행의 기본 원리는 다음과 같다. 쿼리를 완전히 실행하는 대신, 부분적으로 실행하여 이미 얼마나 많은 결과가 나왔는지 추정한다. 예를 들어, 조인 쿼리에서 첫 번째 조인까지만 실행하고, 그 결과의 정확한 개수를 파악한다. 이를 기반으로 다음 조인의 순서를 결정하거나, 나머지 조인이 얼마나 비용이 들 것인지 추정할 수 있다.

구체적인 예를 살펴보자. `SELECT * FROM A JOIN B ON A.id = B.id WHERE A.val > 100` 쿼리를 실행한다고 하자. 전통적인 방식에서는:

1. A에서 `val > 100`인 행의 개수를 추정한다 (예: 1000개로 추정)
2. 각 1000개 행이 B와 조인될 때 얼마나 많은 결과가 나올지 추정한다
3. 이를 기반으로 최적의 실행 순서를 결정한다

ECQO 방식에서는:

1. A에서 `val > 100`인 행들을 실제로 실행한다 (정확한 개수, 예: 950개)
2. 950개 행이 B와 조인될 때 정확히 몇 개의 결과가 나오는지 파악한다 (예: 3000개)
3. 이제 정확한 정보를 기반으로 나머지 계획을 최적화한다

부분 실행의 비용은 제한되어야 한다. 만약 부분 실행이 전체 실행만큼 비용이 많이 들면, ECQO의 의미가 없다. 따라서 Trimmer는 제한된 실행 비용(bounded execution cost) 내에서 최대한의 카디널리티 정보를 얻을 수 있는 방법을 제시한다.

50.3 비용 제한 하에서의 카디널리티 결정

ECQO의 핵심은 비용 제한(cost budget)을 설정하고, 이 제한 내에서 최대한의 카디널리티를 결정하는 것이다. Trimmer는 여러 가지 방법을 제시한다.

첫 번째 방법은 인덱스 기반 샘플링(index-based sampling)이다. 쿼리 조건을 만족하는 행들이 인덱스에서 어떻게 분포되어 있는지 파악하고, 인덱스를 순회하면서 조건을 만족하는 행의 개수를 센다. 인덱스 순회 비용은 풀 테이블 스캔보다 훨씬 저렴할 수 있다.

두 번째 방법은 단계적 실행(progressive execution)이다. 쿼리의 연산 단계를 차례대로 실행하되, 각 단계 후에 진행할지 여부를 결정한다. 예를 들어, 10% 데이터에 대해 쿼리를 실행해보고 그 결과를 기반으로 전체 실행 비용을 추정할 수 있다.

세 번째 방법은 근사 구조(approximation structures)를 사용하는 것이다. 예를 들어, 블룸 필터(Bloom filter)나 count-min sketch 같은 확률적 자료구조를 사용하면, 최소한의 메모리와 계산으로도 근사적인 결과 개수를 빠르게 얻을 수 있다. 이를 정확한 개수로 수렴하도록 반복적으로 개선할 수 있다.

50.4 동적 계획(Dynamic Planning)

ECQO의 또 다른 중요한 측면은 동적 계획(dynamic planning)이다. 전통적인 쿼리 최적화는 static planning으로, 쿼리 시작 전에 전체 실행 계획을 수립한다. 반면 ECQO는 부분 실행의 결과를 기반으로 실행 중에 계획을 수정할 수 있다.

동적 계획의 예를 들면 다음과 같다. 조인 순서를 결정할 때:

1. A JOIN B를 먼저 실행한다 (작은 비용)
2. A JOIN B의 정확한 카디널리티를 파악한다
3. (A JOIN B) JOIN C의 비용을 정확하게 추정한다
4. C JOIN A를 먼저 실행하는 것보다 (A JOIN B) JOIN C가 더 저렴하면 이를 선택한다

이러한 동적 계획은 정적 계획보다 훨씬 더 최적의 실행 계획을 찾을 수 있다. 특히 카디널리티 추정이 부정확한 경우에는 동적 계획의 이점이 매우 크다.

50.5 유사성 쿼리에의 적용

ECQO가 특히 유용한 분야는 유사성 쿼리이다. 벡터 데이터베이스에서 "벡터 q와 거리 d 이내의 모든 벡터를 찾아라"라는 쿼리가 주어졌을 때, 결과가 몇 개일지 미리 추정하는 것은 매우 어렵다.

전통적인 통계 기반 방법은 유사성 쿼리에 적용되기 어렵다:

- 벡터 거리의 분포가 복잡하고 차원에 따라 달라진다
- 벡터 공간의 특성(clustering, manifold structure)을 반영하기 어렵다
- 학습 데이터와 테스트 데이터의 분포 차이가 클 수 있다

ECQO 방식에서는:

1. 각 유사성 쿼리를 실제로 부분 실행한다
2. 인덱스의 일부를 검사하여 현재까지 몇 개의 결과가 나왔는지 파악한다
3. 현재 개수와 검사한 데이터의 비율을 기반으로 전체 결과 개수를 추정한다
4. 이 추정값이 정확한 경우, 이를 최적화 결정에 사용한다

50.6 실험 결과와 성능 특성

Trummer는 전통적인 카디널리티 추정 방법과 ECQO를 비교하는 실험을 수행했다. 결과는 인상적이다.

전통적인 추정의 추정 오류(estimation error)는 평균 5배에서 10배 정도였다. 즉, 실제 결과가 1000개인데 추정값이 5000개 또는 200개인 경우가 흔하다는 뜻이다. 반면 ECQO의 추정 오류는 평균 1.5배 정도로, 매우 정확하다.

쿼리 실행 시간 측면에서도 ECQO는 이점을 보여준다. 정확한 카디널리티 추정으로 인해 더 효율적인 실행 계획을 선택할 수 있고, 이는 전체 쿼리 실행 시간을 단축한다. 복잡한 조인 쿼리의 경우, ECQO를 사용하면 실행 시간을 30% 이상 단축할 수 있다.

부분 실행의 비용도 관리 가능한 수준이다. 일반적으로 부분 실행 비용은 전체 쿼리 실행 시간의 10~20% 정도이므로, 부분 실행으로 인한 오버헤드가 최적화를 통한 이득을 초과하지 않는다.

50.7 본 논문과 Exqutor의 관계

ECQO는 Exqutor의 직접적인 선행 작업이며, Exqutor는 이 개념을 유사성 쿼리에 특화된 형태로 발전시킨다.

Trummer의 ECQO는 부분 실행을 기반으로 한 접근이고, 이는 실무에서 구현하기 위해 여러 가지 오버헤드를 수반한다:

- 각 쿼리마다 부분 실행을 수행해야 한다 (런타임 오버헤드)
- 부분 실행의 결과를 기반으로 정확한 값으로 수렴하는 과정이 복잡하다
- 쿼리 패턴에 따라 부분 실행의 비용이 달라진다

Exqutor는 이러한 문제를 다르게 접근한다:

- 머신러닝 모델을 사용하여 쿼리 특성으로부터 직접 카디널리티를 예측한다
- 모델 학습은 사전에 수행되므로, 런타임 오버헤드가 매우 적다
- 부분 실행 대신, 벡터 검색의 특성(벡터 차원, 거리 임계값, 인덱스 구조 등)을 특징으로 사용한다

특히 유사성 쿼리에서 Exqutor는 다음과 같은 추가적인 고려사항을 반영한다:

- 벡터 공간의 내재적 구조 (manifold structure)
- 쿼리 벡터와 데이터 벡터 간의 거리 분포
- 인덱싱 방법(IVF, HNSW, PQ 등)의 특성
- 임계값(threshold)에 대한 민감도

추가 제기 문제

1. **부분 실행의 오버헤드**: 정확한 카디널리티를 위해 부분 실행을 수행하는 것이 항상 정당한지 의문의 여지가 있다. 간단한 쿼리의 경우 부분 실행의 오버헤드가 이점보다 클 수 있다.
2. **온라인 학습의 필요성**: ECQO는 쿼리마다 부분 실행을 수행하므로, 시간이 지남에 따라 데이터 분포가 변해도 자동으로 적응한다. 하지만 Exqutor 같은 학습 기반 방법은 정기적인 모델 재학습이 필요할 수 있다.
3. **일반화의 한계**: ECQO는 각 쿼리에 특화된 결정을 내리므로 매우 정확하지만, 새로운 패턴의 쿼리에는 적응하기 어렵다. 학습 기반 방법이 더 나은 일반화를 제공할 수 있다.
4. **CAP 트레이드오프**: 정확성(Accuracy), 비용(Cost), 편의성(Practicality) 간의 트레이드오프가 존재한다. ECQO는 정확성을 추구하지만 비용이 크고, 완전 추정 비용이 적지만 정확성이 낮으며, 머신러닝 기반 방법은 중간의 균형을 제공한다.